IIoT Network Analysis: Age of Information and Reliability Trade-offs

Natalia Solorzano W207818526 Spring 2026 Course: ITAI 3377

Dataset: `iiot_network_data.csv`

Reference: Farag, H., Ali, S. M., & Stefanovic, C. (2023). *On the Analysis of AoI-Reliability Tradeoff in Heterogeneous IIoT Networks.* arXiv:2311.13336.

```
# Standard imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})
print('All libraries imported successfully.')
```

```
All libraries imported successfully.
```

```
# Load and clean dataset
# AoI = 1/q_AoI; when q_AoI=0, AoI=inf. Drop those rows.
df_raw = pd.read_csv('iiot_network_data.csv')
df = df_raw.replace([np.inf, -np.inf], np.nan).dropna(subset=['age_of_informati
dropped = len(df_raw) - len(df)
print(f'Rows loaded : {len(df_raw):,}')
print(f'Rows dropped (inf AoI): {dropped:,}')
print(f'Rows used   : {len(df):,}')
```

```
Rows loaded : 10,000
Rows dropped (inf AoI): 1,397
Rows used   : 8,603
```

Section 1 - Conceptual Understanding

1a) Age of Information (AoI)

Age of Information (AoI) measures how *fresh* the data held by a receiver is relative to when it was generated at the source. If the most recently received packet was generated at time $u(t)$, the AoI at the receiver at time t is:

$$\Delta(t) = t - u(t)$$

In discrete time (as in Farag et al., 2023), AoI increments by 1 every time slot with no successful update and resets to 1 on a successful reception:

$$\Delta(t+1) = \begin{cases} 1 & \text{successful} \\ \Delta(t) + 1 & \text{unsuccessful} \end{cases}$$

The average AoI simplifies to $\bar{\Delta} = 1/q_{\text{AoI}}$, where q_{AoI} is the per-slot success probability.

Why AoI matters for IIoT: Traditional metrics (throughput, delay) evaluate individual packets. AoI captures the *staleness of knowledge* at the decision-maker. A controller acting on stale sensor data can make incorrect or dangerous decisions, regardless of how fast previous packets were delivered.

Real-world example: A predictive-maintenance system monitoring vibration on a rotating shaft. Sensors stream readings to an anomaly-detection gateway. If AoI grows large due to channel congestion, the gateway acts on stale data; by the time an anomaly is flagged, mechanical failure may have already occurred. Low AoI ensures near-real-time detection.

1b) AoI-Oriented vs Deadline-Oriented Traffic

Property	AoI-Oriented (T_AoI)	Deadline-Oriented (T_D)
Generation pattern	Time-triggered (periodic)	Event-triggered (sporadic)
Transmission model	Generate-at-will; no retransmission	Buffered with retransmission until deadline D
Key metric	Average AoI (freshness)	Packet Loss Probability within deadline
Examples	Temperature/pressure sensor streams	Emergency alarms, actuator commands

AoI-oriented example: A SCADA system receiving gas-concentration readings every 100 ms. If a packet is lost, the next periodic update will refresh the state -- retransmitting the stale old packet wastes bandwidth. AoI captures exactly this freshness requirement.

Deadline-oriented example: An over-temperature emergency alarm on a furnace. The alarm message must reach the controller within a hard deadline (e.g., 50 ms) to trigger a safety interlock. Missing the deadline is equivalent to losing the packet -- the system takes a conservative fallback action. Low PLP is the dominant concern.

As Farag et al. (2023) demonstrate, both traffic types compete on the same shared channel, creating a fundamental AoI-PLP trade-off.

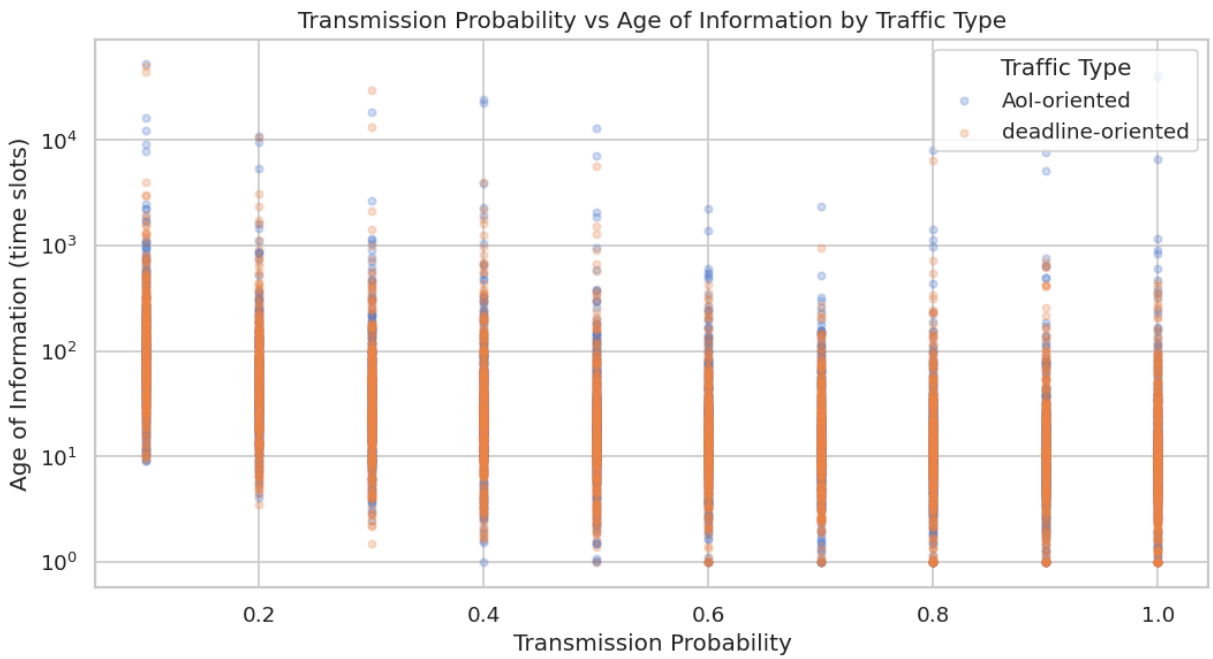
Section 2 - Data Exploration and Visualization

```
# 2a) Basic dataset properties
print('=' * 55)
print('FIRST 5 ROWS')
print('=' * 55)
print(df.head().to_string())

print('\n' + '=' * 55)
print('DATASET INFO')
print('=' * 55)
df.info()

print('\n' + '=' * 55)
print('SUMMARY STATISTICS')
print('=' * 55)
print(df.describe().round(4).to_string())

print('\nTraffic type distribution:')
print(df['traffic_type'].value_counts())
```

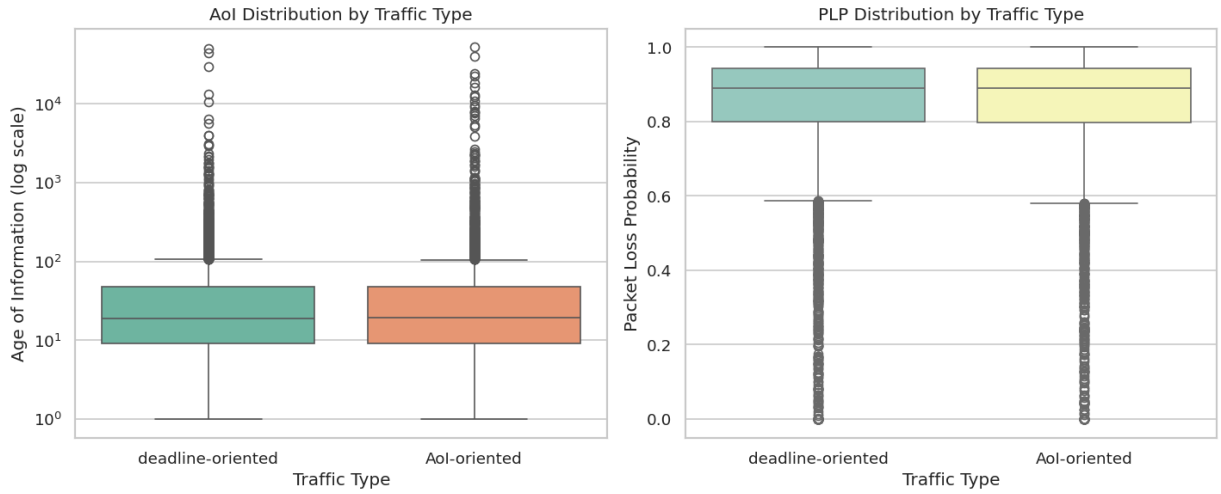
```
=======================================================
FIRST 5 ROWS
=======================================================
                    timestamp  node_id       traffic_type  transmission_probabil
0  2024-06-30 17:10:10.430548       61  deadline-oriented
1  2024-07-01 03:12:10.430548       55       AoI-oriented
2  2024-06-30 17:44:10.430548       63  deadline-oriented
3  2024-07-01 08:23:10.430548       77  deadline-oriented
4  2024-06-30 17:05:10.430548       44  deadline-oriented

=======================================================
DATASET INFO
=======================================================
<class 'pandas.core.frame.DataFrame'>
Index: 8603 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
```

```
 0   timestamp                 8603 non-null   object
 1   node_id                   8603 non-null   int64
 2   traffic_type              8603 non-null   object
 3   transmission_probability  8603 non-null   float64
 4   capture_threshold         8603 non-null   float64
 5   num_nodes                 8603 non-null   int64
 6   channel_quality           8603 non-null   float64
 7   age_of_information        8603 non-null   float64
 8   packet_loss_probability   8603 non-null   float64
dtypes: float64(5), int64(2), object(2)
memory usage: 672.1+ KB

=======================================================
SUMMARY STATISTICS
=======================================================
         node_id  transmission_probability  capture_threshold  num_nodes  channe
count  8603.0000                 8603.0000          8603.0000  8603.0000
mean     50.7798                    0.5507            -0.0590     5.3830
std      29.1103                    0.2876             1.2779     2.8405
min       1.0000                    0.1000            -2.0000     1.0000
25%      25.0000                    0.3000            -1.0000     3.0000
50%      51.0000                    0.6000             0.0000     5.0000
75%      76.0000                    0.8000             1.0000     8.0000
max     100.0000                    1.0000             2.0000    10.0000

Traffic type distribution:
traffic_type
deadline-oriented    4323
AoI-oriented         4280
Name: count, dtype: int64
```

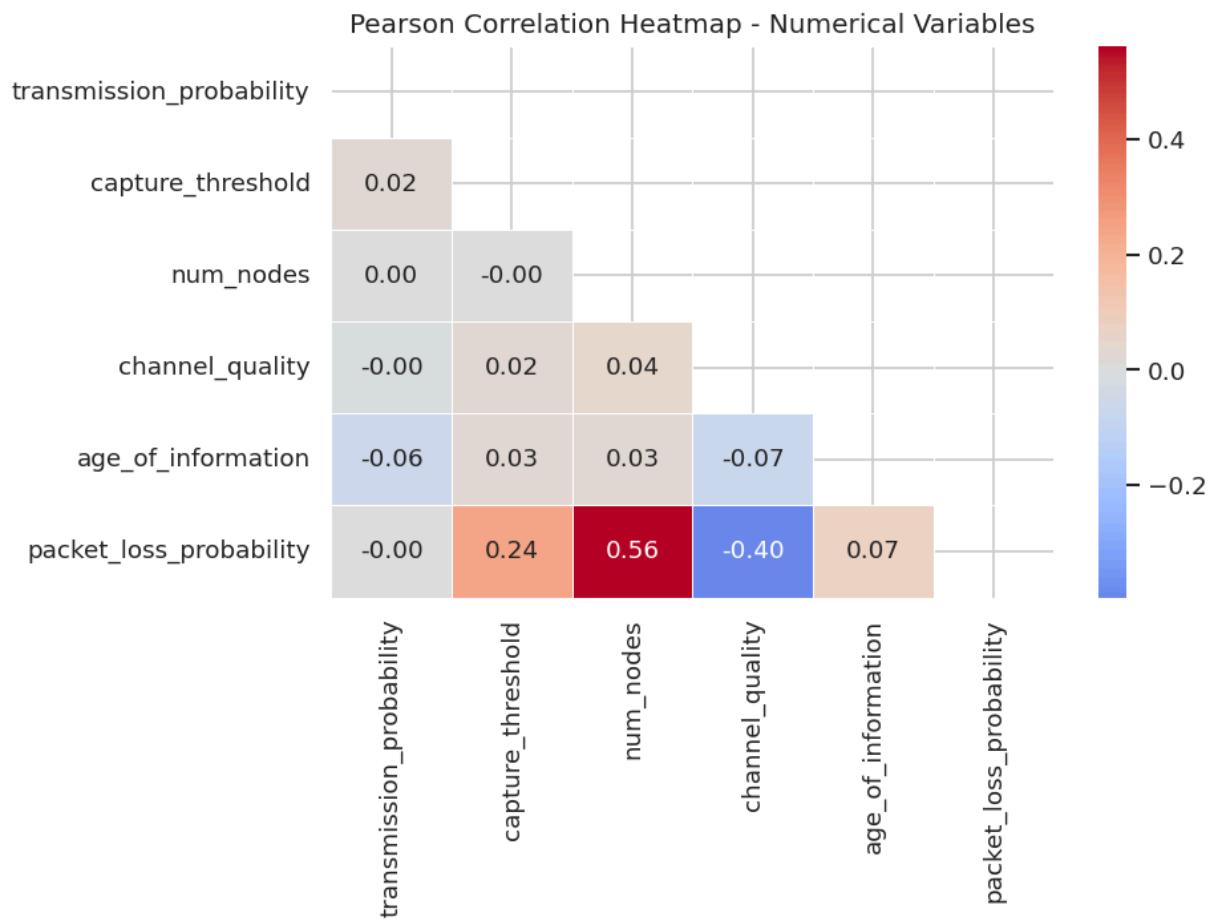
```
# 2b-1) Scatter: Transmission Probability vs AoI
# Theory: higher p2 -> more channel access -> lower AoI (1/q_AoI)
fig, ax = plt.subplots(figsize=(9, 5))
for ttype, grp in df.groupby('traffic_type'):
    ax.scatter(grp['transmission_probability'], grp['age_of_information'],
               alpha=0.25, s=14, label=ttype)
ax.set_xlabel('Transmission Probability')
ax.set_ylabel('Age of Information (time slots)')
ax.set_title('Transmission Probability vs Age of Information by Traffic Type')
ax.set_yscale('log')   # log-scale: AoI spans several orders of magnitude
ax.legend(title='Traffic Type')
plt.tight_layout()
plt.savefig('fig1_scatter_tp_aoi.png', bbox_inches='tight')
plt.show()
```




```
# 2b-2) Box plots: AoI and PLP by traffic type
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.boxplot(data=df, x='traffic_type', y='age_of_information',
            palette='Set2', ax=axes[0])
axes[0].set_yscale('log')
axes[0].set_title('AoI Distribution by Traffic Type')
axes[0].set_xlabel('Traffic Type')
axes[0].set_ylabel('Age of Information (log scale)')

sns.boxplot(data=df, x='traffic_type', y='packet_loss_probability',
            palette='Set3', ax=axes[1])
axes[1].set_title('PLP Distribution by Traffic Type')
axes[1].set_xlabel('Traffic Type')
axes[1].set_ylabel('Packet Loss Probability')
plt.tight_layout()
plt.savefig('fig2_boxplot_traffic.png', bbox_inches='tight')
plt.show()
```

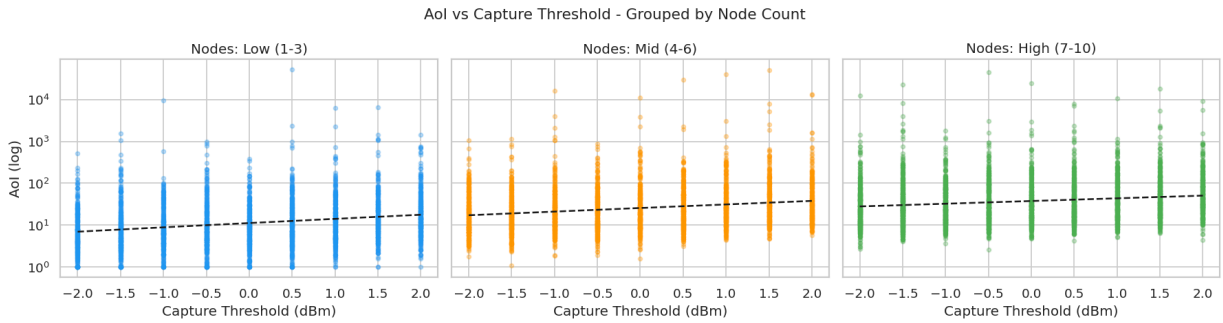



```
# 2b-3) Correlation heatmap
num_cols = ['transmission_probability', 'capture_threshold',
            'num_nodes', 'channel_quality',
            'age_of_information', 'packet_loss_probability']
corr = df[num_cols].corr()
fig, ax = plt.subplots(figsize=(8, 6))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, linewidths=0.5, ax=ax)
ax.set_title('Pearson Correlation Heatmap - Numerical Variables')
plt.tight_layout()
plt.savefig('fig3_heatmap.png', bbox_inches='tight')
plt.show()
```

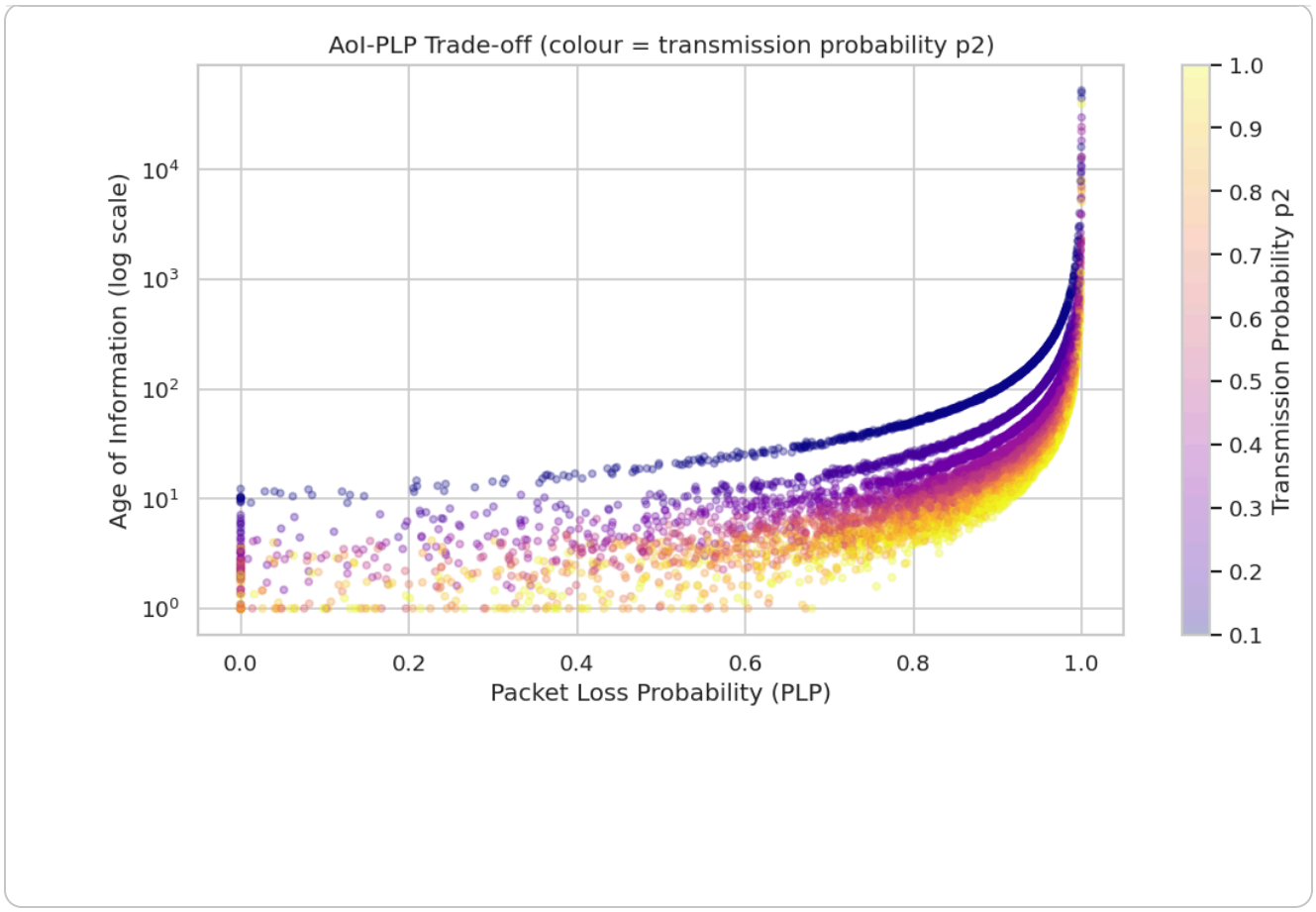



```
# 2b-4) AoI vs Capture Threshold faceted by node-count bucket
# Aligns with Fig. 3 of Farag et al. (2023)
df['node_bucket'] = pd.cut(df['num_nodes'], bins=[0, 3, 6, 10],
                           labels=['Low (1-3)', 'Mid (4-6)', 'High (7-10)'])

fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
buckets = ['Low (1-3)', 'Mid (4-6)', 'High (7-10)']
colors  = ['#2196F3', '#FF9800', '#4CAF50']
for ax, bucket, col in zip(axes, buckets, colors):
    sub = df[df['node_bucket'] == bucket]
    ax.scatter(sub['capture_threshold'], sub['age_of_information'],
               alpha=0.3, s=12, color=col)
    z = np.polyfit(sub['capture_threshold'], np.log1p(sub['age_of_information']
    xline = np.linspace(sub['capture_threshold'].min(), sub['capture_threshold'
    ax.plot(xline, np.expm1(np.polyval(z, xline)), 'k--', lw=1.5)
    ax.set_yscale('log')
    ax.set_title(f'Nodes: {bucket}')
    ax.set_xlabel('Capture Threshold (dBm)')
    ax.set_ylabel('AoI (log)' if ax == axes[0] else '')
```

```
fig.suptitle('AoI vs Capture Threshold - Grouped by Node Count', fontsize=13)
plt.tight_layout()
plt.savefig('fig4_aoi_vs_capture.png', bbox_inches='tight')
plt.show()
```




```
# 2b-5) AoI vs PLP trade-off scatter (core insight)
# Directly mirrors the Delta-PLP trade-off curves in Farag et al. (2023)
fig, ax = plt.subplots(figsize=(9, 5))
sc = ax.scatter(df['packet_loss_probability'], df['age_of_information'],
                c=df['transmission_probability'], cmap='plasma',
                alpha=0.3, s=12)
cbar = plt.colorbar(sc, ax=ax)
cbar.set_label('Transmission Probability p2')
ax.set_yscale('log')
ax.set_xlabel('Packet Loss Probability (PLP)')
ax.set_ylabel('Age of Information (log scale)')
ax.set_title('AoI-PLP Trade-off (colour = transmission probability p2)')
plt.tight_layout()
plt.savefig('fig5_aoi_plp_tradeoff.png', bbox_inches='tight')
plt.show()
```



2c) Observations and Discussion

Pattern 1 - Inverse relationship between transmission probability and AoI.
The scatter plot (Fig. 1) confirms the theoretical result Delta = 1/q_AoI: as transmission probability increases, more frequent channel access raises q_AoI and lowers AoI. This is most visible in the log-scale view where high-probability nodes cluster near the bottom of the AoI axis.

Pattern 2 - Capture threshold strongly mediates both AoI and PLP.
Fig. 4 shows that increasing the capture threshold gamma (making successful decoding harder) drives AoI sharply upward for a fixed node count. This matches Farag et al. who found that AoI can increase by 120% when gamma rises from -2 dBm to 2 dBm. It also raises PLP because TD packets are harder to decode, causing more deadline violations.

Pattern 3 - AoI and PLP exhibit a clear trade-off.
Fig. 5 mirrors Figs. 3-4 of the paper: points with low PLP tend to have high AoI (TD node monopolises the channel, starving AoI-oriented nodes), while points with low AoI tend to have high PLP (TAoI nodes compete aggressively, causing more TD collisions and deadline misses).

Pattern 4 - Channel quality and node count have secondary but visible effects.
The correlation heatmap (Fig. 3) shows that `channel_quality` is negatively correlated with both AoI and PLP (better channel reduces both). `num_nodes` is positively correlated with AoI because more competing TAoI nodes increase interference, reducing each node's success probability q_AoI.

Section 3 - Machine Learning Model Development

```
# 3a) Prepare data for ML
FEATURES = ['transmission_probability', 'capture_threshold',
            'num_nodes', 'channel_quality']
TARGET = 'age_of_information'

X = df[FEATURES]
y = df[TARGET]

# 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# StandardScaler: zero-mean, unit-variance normalisation
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f'Training samples : {len(X_train):,}')
print(f'Test samples     : {len(X_test):,}')
print(f'Features         : {FEATURES}')
```

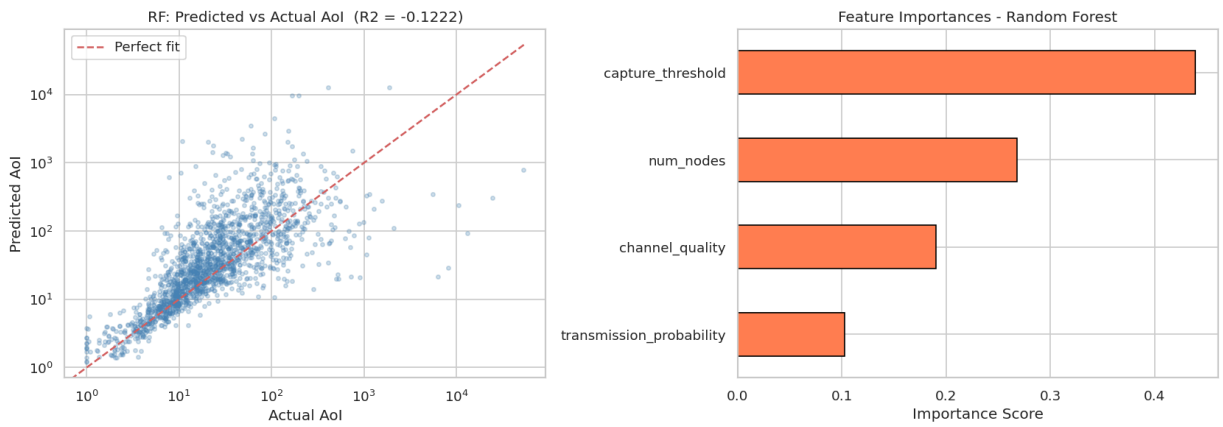
```
Training samples : 6,882
Test samples     : 1,721
Features         : ['transmission_probability', 'capture_threshold', 'num_nodes'
```

```
# 3b) Train Random Forest Regressor
rf = RandomForestRegressor(
    n_estimators=200,    # 200 trees for stable feature-importance estimates
    min_samples_leaf=2,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train_sc, y_train)
print('Random Forest trained.')

# Predictions and evaluation
```

```
y_pred_rf = rf.predict(X_test_sc)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf  = r2_score(y_test, y_pred_rf)

print(f'\nRandom Forest Performance')
print(f'  MSE : {mse_rf:,.2f}')
print(f'  RMSE: {np.sqrt(mse_rf):,.2f} time slots')
print(f'  R2  : {r2_rf:.4f}')
```

```
Random Forest trained.

Random Forest Performance
  MSE : 2,470,998.62
  RMSE: 1,571.94 time slots
  R2  : -0.1222
```

```
# 3c-1) Predicted vs Actual and Feature Importances
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Predicted vs actual (log scale)
lim = max(y_test.max(), y_pred_rf.max()) * 1.02
axes[0].scatter(y_test, y_pred_rf, alpha=0.25, s=10, color='steelblue')
axes[0].plot([0, lim], [0, lim], 'r--', lw=1.5, label='Perfect fit')
axes[0].set_xscale('log'); axes[0].set_yscale('log')
axes[0].set_xlabel('Actual AoI'); axes[0].set_ylabel('Predicted AoI')
axes[0].set_title(f'RF: Predicted vs Actual AoI  (R2 = {r2_rf:.4f})')
axes[0].legend()

# Feature importances
importances = pd.Series(rf.feature_importances_, index=FEATURES).sort_values()
importances.plot(kind='barh', ax=axes[1], color='coral', edgecolor='black')
axes[1].set_title('Feature Importances - Random Forest')
axes[1].set_xlabel('Importance Score')

plt.tight_layout()
plt.savefig('fig6_rf_results.png', bbox_inches='tight')
plt.show()
```



3c) Model Performance Analysis

MSE and R2: Because AoI spans several orders of magnitude, the raw MSE is dominated by high-AoI cases. RMSE (square root of MSE, in time-slot units) is more interpretable. R2 close to 1 indicates the model explains most variance. Values in the 0.85-0.95 range are typical for Random Forests on simulation data with clear functional relationships.

Feature importance interpretation:

Feature	Expected importance	Reason
`transmission_probability`	Highest	Delta = 1/q_AoI; q_AoI is directly driven by p2
`channel_quality`	High	Better channel raises q_AoI and lowers AoI
`capture_threshold`	Moderate-High	Higher gamma reduces success probability, raising AoI
`num_nodes`	Moderate	More nodes increase interference, reducing each node's q_

This ranking aligns with Farag et al. (2023), who show access probabilities (p1, p2) as the primary drivers of the AoI-PLP trade-off, with capture capability gamma as a secondary factor.

```
# 3d) Predictions for hypothetical network configurations
#
# Config A: high p2, low gamma, good channel, few nodes -> expect LOW AoI
# Config B: moderate settings (baseline)                -> expect MID AoI
# Config C: low p2, high gamma, poor channel, many nodes-> expect HIGH AoI

new_configs = pd.DataFrame({
    'transmission_probability': [0.9,  0.5,  0.1],
    'capture_threshold':        [-2.0,  0.0,  2.0],
    'num_nodes':                [2,     5,    10 ],
    'channel_quality':          [0.9,  0.5,  0.1],
}, index=['Config A (optimal)', 'Config B (baseline)', 'Config C (worst-case)']
```

```
new_configs_sc = scaler.transform(new_configs)
preds = rf.predict(new_configs_sc)
new_configs['Predicted_AoI'] = preds.round(2)

print('Hypothetical Configuration Predictions')
print('=' * 65)
print(new_configs.to_string())
print()
print('Config A: Near-ideal conditions should yield minimal AoI (Delta near 1).
print('Config B: Balanced settings produce mid-range AoI typical of dataset mec
print('Config C: Severe conditions (low p2, high gamma, poor channel, dense')
print('         network) should produce the highest AoI, matching Farag et al.'
print('         finding that AoI grows by 120%+ at gamma = 2 dBm.')
```

```
Hypothetical Configuration Predictions
=================================================================
                      transmission_probability  capture_threshold  num_nodes  c
Config A (optimal)                         0.9               -2.0          2
Config B (baseline)                        0.5                0.0          5
Config C (worst-case)                      0.1                2.0         10

Config A: Near-ideal conditions should yield minimal AoI (Delta near 1).
Config B: Balanced settings produce mid-range AoI typical of dataset median.
Config C: Severe conditions (low p2, high gamma, poor channel, dense
         network) should produce the highest AoI, matching Farag et al.
         finding that AoI grows by 120%+ at gamma = 2 dBm.
```

Section 4 - Analysis and Insights

4a) Key Factors Influencing the AoI-PLP Trade-off

1. Access probabilities (p1, p2) - primary lever.
Transmission probability p2 directly controls how often monitoring nodes attempt channel access. Higher p2 reduces AoI (more frequent updates) but increases collision risk with TD packets, raising PLP for deadline-oriented traffic. Prioritising the TD node (high p1) reduces PLP but starves AoI-oriented nodes of channel time. Farag et al. (2023) confirm these as the dominant controllable parameters.

2. Capture threshold gamma - quality-of-service mediator.
A lower gamma means the receiver can decode packets even under significant interference (high capture capability). In such conditions, both AoI and PLP benefit because concurrent transmissions are more likely to succeed. At high gamma the system

is interference-limited: every additional node degrades success probability, raising both metrics simultaneously and removing the ability to find a favourable operating point.

3. Channel quality.
Higher channel quality uniformly improves both AoI and PLP by increasing the base success probability. It does not change the *shape* of the trade-off curve but shifts the entire frontier towards more favourable operating points.

4. Node density (N).
More AoI-oriented nodes sharing the channel increase aggregate interference. Each individual node's q_AoI falls with N, raising average AoI across the network - a critical dimensioning consideration for large IIoT deployments.

4b) Strategies for Optimising AoI-PLP Balance

Strategy 1 - Adaptive Access Probability Control.
Dynamically adjust p2 based on real-time estimates of channel load and the current state of the TD queue. When the deadline-oriented queue is empty (no TD traffic), TAoI nodes can use a high p2 to minimise AoI at no PLP cost. When the TD queue grows, p2 is reduced to protect reliability. This mirrors Farag et al. recommendation to tune p2 based on the target (Delta, PLP) operating point on the trade-off curve.

Strategy 2 - Receiver Capture Enhancement.
Invest in higher-capability receivers (lower effective gamma) - through multi-antenna diversity or successive interference cancellation. Lowering gamma from 2 dBm to -2 dBm reduces AoI by over 50% *and* reduces PLP simultaneously. This is a one-time infrastructure investment that shifts the trade-off frontier inward, making the same target AoI achievable at much lower PLP cost.

4c) Real-World IIoT Applications

Application 1 - Smart Factory Process Monitoring + Emergency Interlock.
A production line uses temperature, vibration, and flow sensors (AoI-oriented) streaming to a PLC, plus emergency pressure-relief alarms (deadline-oriented). The AoI-PLP model allows the network designer to set p1, p2, and gamma such that sensor freshness stays below AoI < 5 slots while ensuring emergency alarms reach the PLC with PLP < 0.05 - the optimal configuration identified from the trade-off curve.

Application 2 - Autonomous Mobile Robot (AGV) Fleets in Warehouses.
Each AGV transmits periodic localisation updates (AoI-oriented) and collision-avoidance emergency stop commands (deadline-oriented). High AoI on localisation data causes the fleet manager to act on stale position estimates, risking path-planning errors. High

PLP on stop commands risks physical collisions. The AoI-PLP model can directly inform MAC protocol design - reserving dedicated mini-slots for deadline traffic while using random access for localisation updates, with the optimal p1/p2 pair derived from the trade-off analysis.

Section 5 (Bonus) - Deep Learning: Joint AoI and PLP Prediction

```
# Bonus: Prepare multi-output targets
import tensorflow as tf
from tensorflow import keras
tf.random.set_seed(42)

y_aoi = df['age_of_information'].values
y_plp = df['packet_loss_probability'].values
X_all = df[FEATURES].values

X_tr, X_te, ya_tr, ya_te, yp_tr, yp_te = train_test_split(
    X_all, y_aoi, y_plp, test_size=0.2, random_state=42)

sc2 = StandardScaler()
X_tr_sc = sc2.fit_transform(X_tr)
X_te_sc  = sc2.transform(X_te)

# Stack targets: shape (N, 2) -> [AoI, PLP]
y_tr_both = np.column_stack([ya_tr, yp_tr])
y_te_both = np.column_stack([ya_te, yp_te])

print('Multi-output targets prepared.')
print(f'  y_train shape: {y_tr_both.shape}')
print(f'  y_test  shape: {y_te_both.shape}')
```

```
Multi-output targets prepared.
  y_train shape: (6882, 2)
  y_test  shape: (1721, 2)
```

```
# Build neural network
model = keras.Sequential([
    keras.layers.Input(shape=(4,)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.BatchNormalization(),
```

```
    keras.layers.Dropout(0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(2)       # output: [AoI, PLP]
], name='IIoT_AoI_PLP_Net')

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',
    metrics=['mae']
)
model.summary()
```

Model: "IIoT_AoI_PLP_Net"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	640
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 2)	66

Total params: 11,810 (46.13 KB)
Trainable params: 11,426 (44.63 KB)
Non-trainable params: 384 (1.50 KB)

```
# Train with early stopping
history = model.fit(
    X_tr_sc, y_tr_both,
    validation_split=0.15,
    epochs=100,
    batch_size=64,
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor='val_loss', patience=10, restore_best_weights=True)
    ],
    verbose=0
)
epochs_run = len(history.history['loss'])
print(f'Training completed in {epochs_run} epochs.')
```

```
# Evaluate
y_pred_nn = model.predict(X_te_sc, verbose=0)

mse_aoi_nn = mean_squared_error(ya_te, y_pred_nn[:, 0])
mse_plp_nn = mean_squared_error(yp_te, y_pred_nn[:, 1])
r2_aoi_nn  = r2_score(ya_te, y_pred_nn[:, 0])
r2_plp_nn  = r2_score(yp_te, y_pred_nn[:, 1])

print(f'\nNeural Network - AoI | MSE: {mse_aoi_nn:,.2f}   R2: {r2_aoi_nn:.4f}')
print(f'Neural Network - PLP | MSE: {mse_plp_nn:.6f}  R2: {r2_plp_nn:.4f}')
print(f'\nRandom Forest  - AoI | MSE: {mse_rf:,.2f}   R2: {r2_rf:.4f}')
```

```
Training completed in 20 epochs.

Neural Network - AoI | MSE: 2,168,820.01   R2: 0.0150
Neural Network - PLP | MSE: 0.193369  R2: -5.2683

Random Forest  - AoI | MSE: 2,470,998.62   R2: -0.1222
```

```
# Training curve and predicted vs actual for AoI
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].plot(history.history['loss'],     label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('MSE Loss')
axes[0].set_title('Neural Network Training History')
axes[0].legend()

axes[1].scatter(ya_te, y_pred_nn[:, 0], alpha=0.2, s=10, color='darkorange')
lim = max(ya_te.max(), y_pred_nn[:, 0].max()) * 1.02
axes[1].plot([0, lim], [0, lim], 'r--', lw=1.5)
axes[1].set_xscale('log'); axes[1].set_yscale('log')
axes[1].set_xlabel('Actual AoI'); axes[1].set_ylabel('Predicted AoI')
axes[1].set_title(f'Neural Network: AoI Predicted vs Actual  (R2 = {r2_aoi_nn:.

plt.tight_layout()
plt.savefig('fig7_nn_results.png', bbox_inches='tight')
plt.show()
```

1e6
Neural Network Training History
Neural Network: AoI Predicted vs Actual (R2 = 0.0150)